第二章 变量、所有权和借用

2.1.1 变量绑定

- 大多数语言中，要么只支持声明可变的变量，要么只支持声明不可变的变量
  - 前者灵活、后者安全
  - C中，只支持声明可变的变量，或用const标识符标为常量

```
const double pi = 3.1415926;
double pii = 3.1415926;
pi = 123.0;
pii = 123.0;
```

- Rust语言既要又要

2.1.1 变量绑定

- C:　　int a = 100; 【变量赋值】
- Rust: let a = 100;　【变量绑定】
- 绑定潜在的意思是在这种变量申明的情况下变量是不可变的

```
fn main() {
    let x = 5;
    println!("The value of x is: {}", x);
    x = 6;
    println!("The value of x is: {}", x);
}
```



2.1.1 变量绑定

- 默认无法改变的好处在于能够减少“多处使用”带来的错误。
- 请看这一段典型的容易出错的C代码:

```
int fun() {
   int tmp;

   while(...) {
        xxx = tmp;
        tmp = yyy;
   }

   ......

   while(...) {
        xxx = tmp;
        tmp = yyy;
   }
}
```

2.1.1 变量绑定

- 怎样申请能够变化的“变量”：只要在变量名前加一个 mut标识

```
fn main() {
    let mut x = 5;
    println!("The value of x is: {}", x);
    x = 6;
    println!("The value of x is: {}", x);
}
```

- 一个变量到底要不要可变取决于程序员，且Rust和C在这里的逻辑相反。
- 为了更好的可读性，Rust也支持Const标识

2.1.2 变量解构*

- let解构：从一个相对复杂的变量中，匹配出该变量的一部分内容：

```
fn main() {
    let (a, mut b): (bool,bool) = (true, false);
    // a = true,不可变; b = false, 可变
    println!("a = {:?}, b = {:?}", a, b);

    b = true;
    assert_eq!(a, b);
}
```

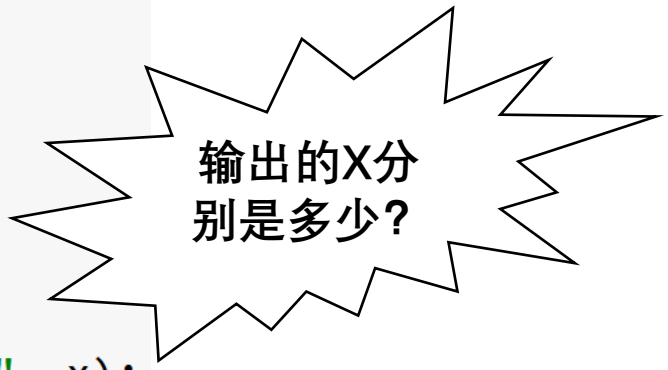
2.1.3 变量遮蔽（Shadowing）*

- Rust 允许声明相同的变量名，在后面声明的变量会遮蔽掉前面声明的

```rust
fn main() {
    let x = 5;
    // 在main函数的作用域内对之前的x进行遮蔽
    let x = x + 1;

    {
        // 在当前的花括号作用域内，对之前的x进行遮蔽
        let x = x * 2;
        println!("The value of x in the inner scope is: {}", x);
    }

    println!("The value of x is: {}", x);
}
```



2.2 基本类型

- 变量的类型决定了它占用的内存大小和数据表达形式
- C语言中：

```
char
unsigned char
int
float
double
```

```
enum{}
struct{}
```

- Rust变量类型分为基本类型、复合类型。

2.2 基本类型

- 基本类型：
  - 数值类型：有符号整数 (i8, i16, i32, i64, isize)、 无符号整数 (u8, u16, u32, u64, usize) 、浮点数 (f32, f64)、以及有理数、复数
  - 字符串：字符串字面量和字符串切片 &str
  - 布尔类型：true 和 false
  - 字符类型：表示单个 Unicode 字符，存储为 4 个字节（和C的差异？）
  - 单元类型：即 ()，其唯一的值也是 ()

2.2.1 数值类型

- 整数类型：
- i 是英文单词 integer 的首字母，与之相反的是 u，代表无符号 unsigned 类型
- 用1位二进制代表符号

长度	有符号类型	无符号类型
8 位	`i8`	`u8`
16 位	`i16`	`u16`
32 位	`i32`	`u32`
64 位	`i64`	`u64`
128 位	`i128`	`u128`
视架构而定	`isize`	`usize`

2.2.1 数值类型

- 默认类型问题：
  - 在前面的例子中，let mut x = 5; 这一行代码并没有指定变量的类型，是否没有类型？
  - Rust 整型默认使用 i32，即上述 x 变量默认为i32类型

  - 指定类型时使用：
```
let a : u8 = 255;
```
    - 注意冒号

2.2.1 数值类型

- 整数溢出的问题：看下面这段代码。
  - println! (“{}”, x)类似于C语言的printf，花括号代表占位符，能够自动匹配类型。

```
let mut x : u8 = 255;
x = x + 1;
println!("{}", x);
```



- 两种情况会发生：
  - 默认情况下（Debug模式），编译报错!
  - Release模式下，输出为0

2.2.1 数值类型

- 浮点类型
  - 和C语言类似，分为单精度（f32）和双精度（f64）
  - 小数默认情况下为f64

```
fn main() {
    let x = 2.0; // f64

    let y: f32 = 3.0; // f32
}
```

2.2.1 数值类型

- 和C语言类似，使用浮点数要小心
- 浮点数往往是你想要数字的近似表达
  - 浮点数避免在浮点数上测试相等性

```
fn main() {
    // 断言0.1 + 0.2与0.3相等
    assert!(0.1 + 0.2 == 0.3);
}
```

assert!()用于在程序运行时检查特定条件是否为真

  - 如果非要判断的话应该怎么写代码?
- 浮点数有一些特殊值：NaN (not a number) 等

2.2.1 数值类型——数字运算

- 基本运算与C语言一致，包括加减乘除求余、按位运算、逻辑运算

```
fn main() {
    // 加法
    let sum = 5 + 10;

    // 减法
    let difference = 95.5 - 4.3;

    // 乘法
    let product = 4 * 30;

    // 除法
    let quotient = 56.7 / 32.2;

    // 求余
    let remainder = 43 % 5;
}
```

运算符	说明
& 位与	相同位置均为1时则为1，否则为0
| 位或	相同位置只要有1时则为1，否则为0
^ 异或	相同位置不相同则为1，相同则为0
! 位非	把位中的0和1相互取反，即0置为1，1置为0
<< 左移	所有位向左移动指定位数，右位补0
>> 右移	所有位向右移动指定位数，带符号移动（正数补0，负数补1）

2.2.1 数值类型——序列

- Rust 提供了一个非常简洁的方式，用来生成连续的数值，例如 1..5，生成从 1 到 4 的连续数字，不包含 5；1..=5，生成从 1 到 5 的连续数字，包含 5，它的用途很简单，常常用于循环中：

```
for i in 1..=5 {
    println!("{}",i);
}
```

2.2.2 字符、布尔、单元类型

- 字符类型
  - 和C语言原理一样，但又有很大差别。看下面这段代码：

```
fn main() {
    let c = 'z';
    let z = 'ℤ';
    let g = '国';
    let heart_eyed_cat = '😻';
}
```

2.2.2 字符、布尔、单元类型

- 回忆一下C语言里面，字符类型数据数字和字符的对应：ASCII码表

- 很明显，Rust能够表示的字符更多。
  - Unicode：旨在表示世界所有字符
  - 一个字符占4个字节

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99
0010 0100	36	24	$	0100 0100	68	44	D	0110 0100	100
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105
			*	0100 1010	74	4A	J	0110 1010	106
			+	0100 1011	75	4B	K	0110 1011	107
			,	0100 1100	76	4C	L	0110 1100	108
			-	0100 1101	77	4D	M	0110 1101	109
			.	0100 1110	78	4E	N	0110 1110	110
			/	0100 1111	79	4F	O	0110 1111	111

```rust
fn main() {
    let x = '中';
    println!("字符'中'占用了{}字节的内存大小", size_of_val(&x));
}
```

2.2.2 字符、布尔、单元类型

- 布尔类型（bool）
  - 两个可能的值：true 和 false
  - 占1个字节

```
fn main() {
    let t = true;

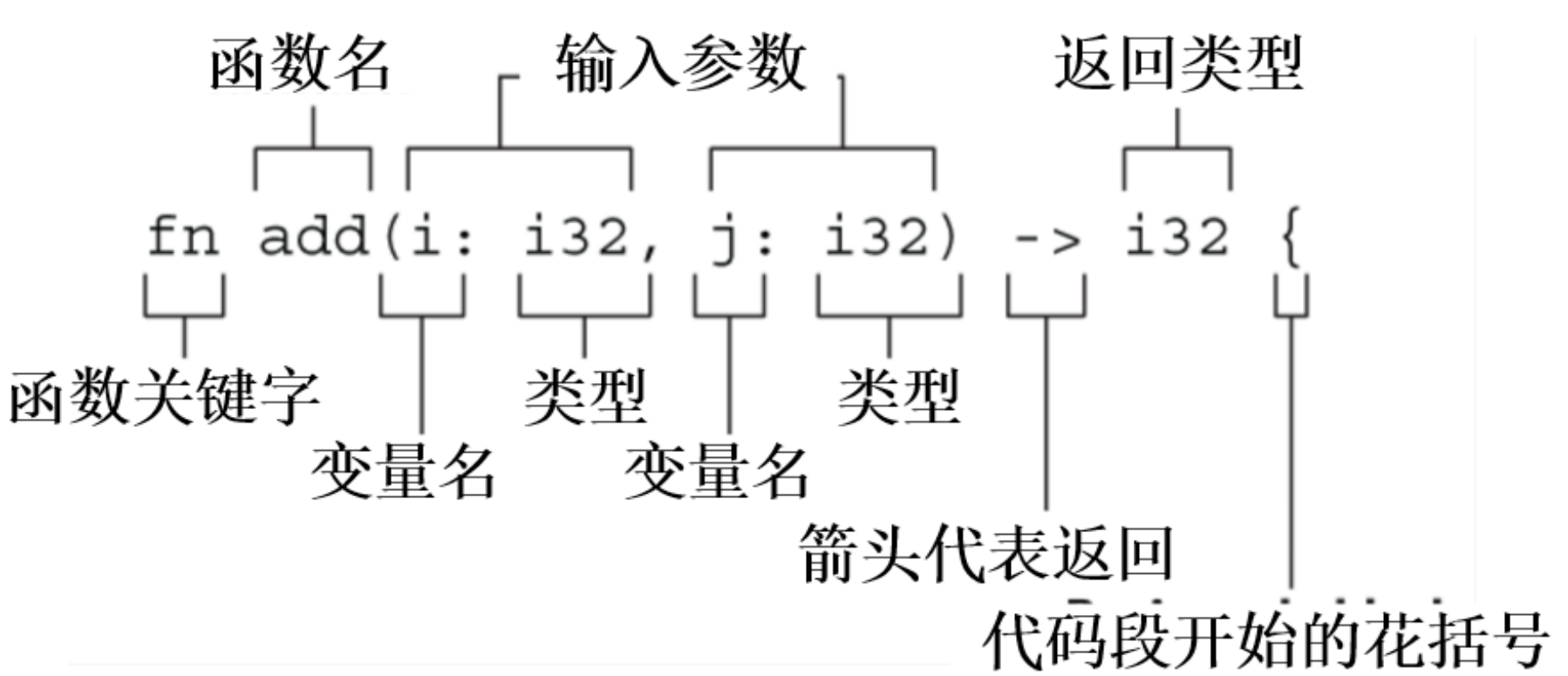
    let f: bool = false; // 使用类型标注,显式指定f的类型

    if f {
        println!("这是段毫无意义的代码");
    }
}
```

- 和C语言有一定的区别
  - C中0、1分别对应false、true
  - Rust不行（编译会报错）
  - 从这里也能看出C的灵活和Rust的安全倾向

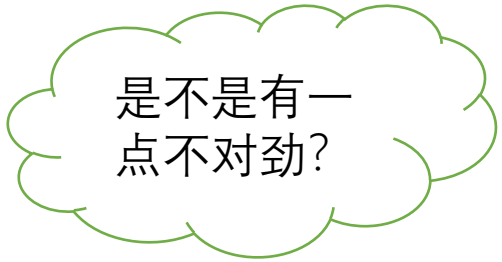
2.2.3 函数

- Rust中的函数和其他语言基本一致



2.2.3 函数

- 来看一段代码：




```
fn add(i: i32, j: i32) -> i32 {
    i + j
}
```

- Return？ 分号？
- 语句和表达式，在C语言中是不进行区分的

2.2.3 函数——语句和表达式

- Rust 的函数体是由一系列语句组成，最后由一个表达式来返回值，例如：

```
fn add_with_extra(x: i32, y: i32) -> i32 {
    let x = x + 1; // 语句
    let y = y + 5; // 语句
    x + y // 表达式
}
```

- Rust明确区分**语句（statement）和表达式（expression）**
  - 语句会执行一些操作但是不会返回一个值，而表达式会在求值后返回一个值

2.2.3 函数——语句和表达式

- 语句

```
let a = 8;
let b: Vec<f64> = Vec::new();
let (a, c) = ("hi", false);
```

  - 以上都是语句，它们完成了一个具体的操作，但是并没有返回值，因此是语句。
  - 如下形式是错误的，为什么？

```
let b = (let a = 8);
```

let是表达式，是一种操作，没有返回值

2.2.3 函数——语句和表达式

- 表达式
  - 表达式会进行求值，然后返回一个值。例如 5 + 6，在求值后，返回值 11，因此它就是一条表达式
  - 表达式可以成为语句的一部分，例如 let y = 6 中，6 就是一个表达式（求值后返回一个值 6）
  - 调用函数等，都是表达式。总之，能返回值，它就是表达式

```
fn main() {
    let y = {
        let x = 3;
        x + 1
    };

    println!("The value of y is: {}", y);
}
```

用花括号括起来的语句！

```
{
    let x = 3;
    x + 1
}
```

2.2.3 函数——语句和表达式

- 在前面的例子中都有一个特点：表达式的最后一行没有分号
- 如果在表达式后加分号，它就会变成语句，且**不会**返回值
- 表达式隐式地返回一个 ()，可理解为NULL。比如下面这段代码

```rust
fn main() {
    assert_eq!(ret_unit_type(), ())
}

fn ret_unit_type() {
    let x = 1;
    // if 语句块也是一个表达式，因此可以用于赋值，也可以直接返回
    // 类似三元运算符，在Rust里我们可以这样写
    let y = if x % 2 == 1 {
        "odd"
    } else {
        "even"
    };
    // 或者写成一行
    let z = if x % 2 == 1 { "odd" } else { "even" };
}
```

返回到变量y了

2.2.3 函数——函数参数

- Rust 是静态类型语言，因此需要你为每一个函数参数都标识出它的具体类型，例如：

```
fn main() {
    another_function(5, 6.1);
}

fn another_function(x: i32, y: f32) {
    println!("The value of x is: {}", x);
    println!("The value of y is: {}", y);
}
```

- 去掉 x 或者 y 的任何一个的类型，都会报错

2.2.3 函数——函数返回

- 函数的返回值就是函数体**最后**一条表达式的返回值，也可以用 return
  - 结合例子，怎样理解“最后”？
- 来看一段代码，同时使用 return 和表达式作为返回值：

```
fn plus_or_minus(x:i32) -> i32 {
    if x > 5 {
        return x - 5
    }

    x + 5
}

fn main() {
    let x = plus_or_minus(5);

    println!("The value of x is: {}", x);
}
```

2.2.3 函数——函数返回 () 即NULL

- 请注意这里不是笔误，就是一组空的括号。
- 单元类型 ()，是一个零长度的元组，理解为NULL
  - 函数没有返回值，那么返回一个 ()
  - 通过 ; 结尾的语句返回一个 ()
- 假如你的函数需要返回一个 u32 值，但是你以表达式; 的语句形式作为函数的最后一行代码，就会报错:

```
fn add(x:u32,y:u32) -> u32 {
    x + y;
}
```

2.3.1 所有权

- 一段不安全的C代码：

```
int* foo() {
    int a;              // 变量a的作用域开始
    a = 100;
    char *c = "xyz";    // 变量c的作用域开始
    return &a;
}                       // 变量a和c的作用域结束
```



- 变量a离开作用域被释放，指针悬空!
- 已经不用的内存区域如何处理?

2.3.1 所有权

- **垃圾回收机制(GC)**，在程序运行时不断寻找不再使用的内存，典型代表：Java、Go
- **手动管理内存的分配和释放**，在程序中，通过函数调用的方式来申请和释放内存，典型代表：C\C++：malloc、free
  - **编译时通常不会报错**!
- **通过所有权来管理内存**，编译器在编译时会根据一系列规则进行检查：Rust

2.3.1 所有权

- 一段不安全的C代码（2）：

```
int fun() {
    char s[10] = "hello!";
    char * p1;
    char * p2;
    p1 = s;
    p2 = s;
    free(p1);
    *p2 = "world!";
    free(p2);
}
```



- 指针p1，p2同时指向 s ， 内存泄漏、二次释放

2.3.1 所有权——栈(Stack)与堆(Heap)

- 栈和堆：为程序在运行时提供可供使用的内存空间
- 栈：后进先出
  - 增加数据叫做**进栈**，移出数据则叫做**出栈**
  - 栈中的所有数据都必须占用**已知且固定大小**的内存空间
- 堆：可以随意放置和取出的空间
  - 可以放置大小未知或者可能变化的数据
  - 分配内存：需求——找到空间——返回地址
  - 天然没有组织，容易出问题

2.3.1 所有权——变量作用域

- 作用域是一个变量在程序中有效的范围，假如有这样一个变量：

```
{                              // s 在这里无效，它尚未声明
    let s = "hello";           // 从此处起，s 是有效的

    // 使用 s
}                              // 此作用域已结束，s不再有效
```

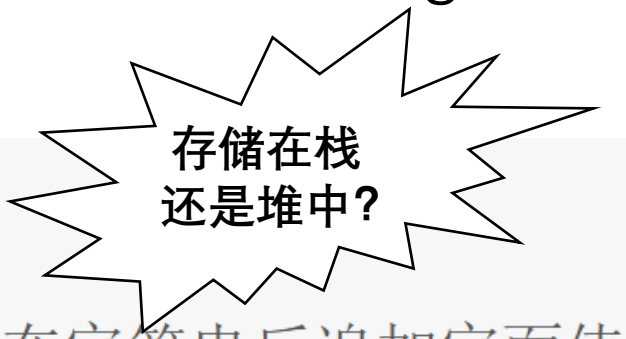
- 此时，s 是被硬编码进程序里的字符串值，无法修改
- 离开作用域后，变量被自动释放

2.3.1 所有权——String类型

- 类似于C语言中的字符串数组，或STL库中的string容器。可动态修改值、可动态调整容量。

```
let s = String::from("hello");
```

- 和C++类似，：：是一种调用操作符，这里表示调用 String 类型中的 from 关联函数




```
let mut s = String::from("hello");

s.push_str(", world!"); // push_str() 在字符串后追加字面值

println!("{}", s); // 将打印 `hello, world!`
```

2.3.1 所有权——基本含义

- 一段不安全的C代码（2）：

```
int fun() {
    char s[10] = "hello!";
    char * p1;
    char * p2;
    p1 = s;
    p2 = s;
    free(p1);
    *p2 = "world!";
    free(p2);
}
```

- 这个例子中的所有权

2.3.1 所有权——基本含义

- 再来看一段代码：

```
let x = 5;
let y = x;
```

  - 这段代码有没有发生所有权的变化？
- 这个呢？

```
let s1 = String::from("hello");
let s2 = s1;
```

- String类型是一个复杂类型，由存储在栈中的**堆指针**、**字符串长度**、**字符串容量**共同组成
  - 指向了一个堆上的空间，这里存储着它的真实数据。要全部拷贝开销较大

2.3.1 所有权——基本含义

- 拷贝String的案例中，如果仅拷贝栈上的指针，开销很小但堆上的内容会有两个“所有者”：s1、s2
  - 当离开作用域时自动释放内存，出现二次释放问题
- Rust的解决：**一个值只允许有一个所有者**
  - 当所有权从s1转移给了s2（赋值操作），s1随即失效

- 所有权转移后再来使用旧的所有者，会怎样？看一个例子：

```rust
let s1 = String::from("hello");
let s2 = s1;

println!("{}, world!", s1);
```



- Rust编译报错：

```
error[E0382]: borrow of moved value: `s1`
 --> src/main.rs:6:24
  |
3 | let s1 = String::from("hello");
  |     -- move occurs because `s1` has type `String`, which
4 | let s2 = s1;
  |          -- value moved here
5 |
6 | println!("{}, world!", s1);
  |                        ^^ value borrowed here after move
  |
```



2.3.1 所有权——基本含义

- Rust所有权规则小结：

1. Rust 中每一个值都被一个变量所拥有，该变量被称为值的所有者
2. 一个值同时只能被一个变量所拥有，或者说一个值只能拥有一个所有者
3. 当所有者（变量）离开作用域范围时，这个值将被丢弃(drop)

2.3.1 所有权——Rust中的移动（move）

- **浅拷贝(shallow copy) 和 深拷贝(deep copy)、移动(move)**
  - 浅拷贝：只拷贝复杂结构中的**栈**存储内容，开销小
  - 深拷贝：拷贝复杂结构中**堆、栈**的所有存储内容，开销大
  - C/C++都有浅拷贝+深拷贝机制
- Rust浅拷贝后使被拷对象失效，可以理解为移动
  - **默认Rust对基本类型直接拷贝，对复杂变量进行移动操作即所有权转移**
  - 如果一定要深拷贝则需要使用对象内置函数clone()：

```
let s1 = String::from("hello");
let s2 = s1.clone();

println!("s1 = {}, s2 = {}", s1, s2);
```

2.3.1 所有权——移动（move）

- 几个例子：
- C++：

```
string s1 = "hello";
string s2 = s1;

char *s4 = (char*)malloc(10);
char *s5 = s4;


char s6[10] = "hello";
char *s7;
s7 = s3;

char s8[10];
strcpy(s8, s7);
```



2.3.1 所有权——函数传值与返回

- 先回忆C语言中的情况：值传递和引用传递（地址传递）
- Rust中，函数传值和返回一样会发生 移动 或者 复制
  - 遵循基本变量复制，复杂变量移动的规则
- 看下面这个例子：

```rust
fn main() {
    let s = String::from("hello");  // s 进入作用域

    takes_ownership(s);             // s 的值移动到函数里 ...
                                    // ... 所以到这里不再有效

    let x = 5;                      // x 进入作用域

    makes_copy(x);                  // x 应该移动函数里，
                                    // 但 i32 是 Copy 的，所以在后面可继续使用 x

} // 这里，x 先移出了作用域，然后是 s。但因为 s 的值已被移走，
  // 所以不会有特殊操作

fn takes_ownership(some_string: String) { // some_string 进入作用域
    println!("{}", some_string);
} // 这里，some_string 移出作用域并调用 `drop` 方法。占用的内存被释放

fn makes_copy(some_integer: i32) { // some_integer 进入作用域
    println!("{}", some_integer);
} // 这里，some_integer 移出作用域。不会有特殊操作
```

2.3.2 引用与借用

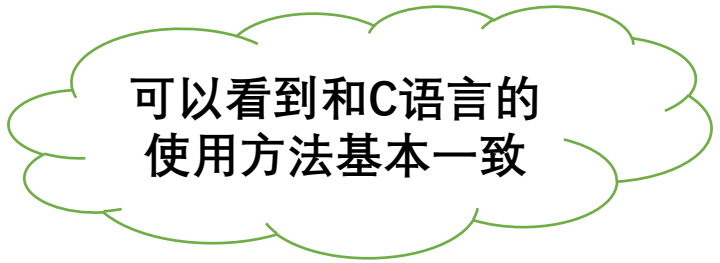
- 所有权很强大，避免了内存的不安全性，但是也带来了一个新麻烦： **总是把一个值传来传去来使用它，复杂**
- 在C语言中有指针和引用的概念通过地址来直接使用变量：
  - int *p; &a;
- Rust 通过“借用(Borrowing)”这个概念来达成，**获取变量的引用，称之为借用**

2.3.2 引用与借用

- 常规引用是一个指针类型，指向了对象存储的内存地址。在下面代码中，我们创建一个 i32 值的引用 y，然后使用解引用运算符来解出 y 所使用的值:

```
fn main() {
    let x = 5;
    let y = &x;

    assert_eq!(5, x);
    assert_eq!(5, *y);
}
```



2.3.2 引用与借用

- 对于函数传参：

```
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("The length of '{}' is {}.", s1, len);
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

- 无需像上节一样：先通过函数参数传入所有权，然后再通过函数返回来传出所有权，代码更加简洁
- calculate_length 的参数 s 类型从 String 变为 &String
- 分为**不可变引用**和**可变引用**

要注意区分String、&str、&String

2.3.2 引用与借用

```
fn main() {
    let s = String::from("hello");

    change(&s);
}

fn change(some_string: &String) {
    some_string.push_str(", world");
}
```

编译报错

```
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

变量申明、可变引用创建、
形式参数有没有加mut决定
是否能修改引用值

2.3.2 引用与借用——可变引用的使用条件

- **条件1：同一作用域，特定数据只能有一个可变引用**：

```
let mut s = String::from("hello");

let r1 = &mut s;
let r2 = &mut s;

println!("{}, {}", r1, r2);
```



- 麻烦，但安全，编译时即避免：
  - 两个或更多的指针同时访问同一数据
  - 至少有一个指针被用来写入数据
  - 没有同步数据访问的机制

2.3.2 引用与借用——可变引用的使用条件

- **条件2：同一对象的可变引用和不可变引用不能同时存在：**

```
let mut s = String::from("hello");

let r1 = &s; // 没问题
let r2 = &s; // 没问题
let r3 = &mut s; // 大问题

println!("{}, {}, and {}", r1, r2, r3);
```



2.3.2 引用与借用——可变引用的使用条件

- **条件3：不能有悬垂引用(Dangling References)**
- 悬垂引用也叫做悬垂指针，意思为指针指向某个值后，这个值被释放掉了，而指针仍然存在

```
fn main() {
    let reference_to_nothing = dangle();
}

fn dangle() -> &String {
    let s = String::from("hello");

    &s
}
```

